

TRAINEE LAB GUIDE

Shopify Agentic AI Developer Bootcamp — Koenig Solutions for V Group Inc.

Lab 3.8

Webhooks & Event-Driven Foundations (Node.js)

Module	3 — Shopify Platform, Storefront API, Admin API & Native AI
Topic	3.8 — a lightweight webhook receiver, registered and triggered against a real dev store
Estimated time	30–40 minutes
Tools needed	Terminal, Node.js 20+, a code editor, a tunnel tool (e.g. ngrok or Cloudflare Tunnel)
Prerequisite	A Shopify dev store and a Dev Dashboard custom app with its Client ID/Secret in hand (the same app from Lab 3.2 works — its Client Secret is also the webhook signing key)

What You're Building

GOAL

Everything so far in Module 3 has been the agent reaching out to Shopify. This lab flips that: a small Express server that Shopify calls into the moment a real order is placed in your dev store — the foundation Module 5's event-driven agents build directly on.

Shopify can't reach a plain `localhost` address, so this lab also covers exposing your local server through a public tunnel — the same technique you'd use for any local webhook development, Shopify or otherwise.

What the receiver does

- Verifies every request's `X-Shopify-Hmac-Sha256` signature before trusting anything in it
- Responds `200` immediately, deferring any real processing until after the response is sent
- Logs the order details once a request passes verification

Prefer to vibe-code this instead? The Module 3 page has a ready-to-use Claude Code prompt covering this same build. This guide gives you the concrete code either way — to type yourself, or to compare against what Claude Code produces.

Steps 1–2: Project Setup and a Public Tunnel

1 Create the project and install dependencies

TERMINAL

```
mkdir shopify-webhook-receiver
cd shopify-webhook-receiver
npm init -y
npm pkg set type=module
npm install express dotenv
```

.ENV

```
SHOPIFY_STORE_DOMAIN=your-store.myshopify.com
SHOPIFY_CLIENT_ID=your-client-id
SHOPIFY_CLIENT_SECRET=your-client-secret
```

.GITIGNORE

```
node_modules
.env
```

Reusing Lab 3.2's app: this is the same Dev Dashboard custom app and Client ID/Secret from Lab 3.2 — no new app registration needed. Its Client Secret does double duty here as the key Shopify signs every webhook payload with.

2 Expose your local server with a tunnel

Shopify's servers need a real public HTTPS URL to call — they can't reach `localhost`. A tunnel tool gives your machine a temporary public address that forwards straight to your local port.

TERMINAL (AFTER INSTALLING NGROK, OR USE ANY EQUIVALENT TUNNEL TOOL)

```
ngrok http 3000
```

Copy the `https://...ngrok-free.app` URL it prints — you'll use it as the webhook's callback URL in Step 5. Leave this running for the rest of the lab.

The URL changes every restart on a free ngrok account. If you stop and restart the tunnel, re-register the webhook in Step 5 with the new URL.

Steps 3–4: Build and Register the Receiver

3 Write the receiver

The one non-obvious requirement: the HMAC has to be recomputed over the exact raw request bytes Shopify sent, not a re-serialized JSON object — so this route reads the body as a raw buffer instead of using `express.json()`.

SERVER.JS

```
import "dotenv/config";
import express from "express";
import crypto from "crypto";

const app = express();
const WEBHOOK_SECRET = process.env.SHOPIFY_CLIENT_SECRET;

function verifyShopifyWebhook(req, res, next) {
  const hmacHeader = req.get("X-Shopify-Hmac-Sha256");
  const digest = crypto
    .createHmac("sha256", WEBHOOK_SECRET)
    .update(req.body)
    .digest("base64");

  let isValid = false;
  try {
    isValid =
      !!hmacHeader &&
      crypto.timingSafeEqual(Buffer.from(digest), Buffer.from(hmacHeader));
  } catch {
    // timingSafeEqual throws if the two buffers differ in length
    // (e.g. a missing or malformed header) - treat that as "invalid", not a crash
    isValid = false;
  }

  if (!isValid) {
    return res.status(401).send("Invalid signature");
  }
  next();
}

app.post(
  "/webhooks/orders-create",
  express.raw({ type: "application/json" }),
  verifyShopifyWebhook,
  (req, res) => {
    // Respond fast - Shopify expects a 200 within a few seconds, or it
    // will retry and can eventually disable the webhook after repeated failures
    res.status(200).send("OK");

    const order = JSON.parse(req.body.toString("utf8"));
    setImmediate(() => {
```

```

    console.log(`Order received: #${order.id} - ${order.total_price} ${order.currency}`);
    // Any slower downstream work (a DB write, another API call) belongs
    // here, after the response has already gone out.
  });
}
);

app.listen(3000, () => console.log("Webhook receiver listening on :3000"));

```

A real, tested pitfall: `express.json()` parses the body into a plain object before your handler ever sees it — recomputing the HMAC over `JSON.stringify()` of that object almost never byte-matches Shopify's original request body (different key order, spacing, or number formatting), so every signature silently fails. Using `express.raw({ type: "application/json" })` on this route keeps `req.body` as the untouched raw buffer the signature was actually computed over.

A second real, tested pitfall: `crypto.timingSafeEqual` throws if its two buffers are different lengths — which happens for free the moment a request arrives with no signature header at all, or a malformed one. Left unguarded, that's an unhandled exception that can crash the process on the very first bad request (a health-check ping, a scanner, anyone). The `try/catch` above turns that case into a normal `401` instead of a crash — verified by sending a deliberately short, malformed signature header and confirming the server stayed up and still returned `401`.

4 Register the webhook against your dev store

A one-off script, reusing the same Admin client-credentials token exchange from Lab 3.2/3.6:

REGISTER-WEBHOOK.JS

```

import "dotenv/config";

const domain = process.env.SHOPIFY_STORE_DOMAIN;
const clientId = process.env.SHOPIFY_CLIENT_ID;
const clientSecret = process.env.SHOPIFY_CLIENT_SECRET;

// Paste the https://...ngrok-free.app URL from Step 2
const CALLBACK_URL = "https://your-tunnel-url.ngrok-free.app/webhooks/orders-create";

const tokenRes = await fetch(`https://${domain}/admin/oauth/access_token`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ client_id: clientId, client_secret: clientSecret, grant_type:
"client_credentials" }),
});
const { access_token } = await tokenRes.json();

const res = await fetch(`https://${domain}/admin/api/2026-07/graphql.json`, {
  method: "POST",
  headers: { "Content-Type": "application/json", "X-Shopify-Access-Token": access_token },
  body: JSON.stringify({

```

```
query: `
  mutation registerWebhook($topic: WebhookSubscriptionTopic!, $sub: WebhookSubscriptionInput!) {
    webhookSubscriptionCreate(topic: $topic, webhookSubscription: $sub) {
      webhookSubscription { id }
      userErrors { field message }
    }
  }
`,
variables: {
  topic: "ORDERS_CREATE",
  sub: { callbackUrl: CALLBACK_URL, format: "JSON" },
},
}),
});
console.log(JSON.stringify(await res.json(), null, 2));
```

TERMINAL

```
node register-webhook.js
```

A successful response shows a `webhookSubscription` with an `id` and an empty `userErrors` array.

Run this against your own dev store to actually confirm it. The receiver's HMAC and fast-response logic above were run and verified locally with simulated requests; this registration call and the real event it produces can only be verified against your own store's credentials and tunnel URL — run it yourself and confirm the same success response before moving on.

Step 5: Trigger a Real Event

5 Start the receiver and place a test order

TERMINAL (SEPARATE FROM THE TUNNEL)

```
node server.js
```

With `ngrok http 3000` still running in another terminal, place a real order in your dev store (a test order via the store's checkout works fine). Watch the receiver's terminal:

EXPECTED OUTPUT

```
Webhook receiver listening on :3000
Order received: #450789469 - 398.00 USD
```

The line appears within a second or two of the order completing — Shopify calls your tunnel URL, the tunnel forwards to `:3000`, the signature checks out, and the receiver logs the order.

What was actually tested vs. what you're confirming live: a valid signature returning `200` and logging the order, a tampered or wrong-secret signature returning `401`, a missing header returning `401`, a malformed header returning `401` without crashing the process, and the response returning in well under a second even when the simulated downstream work took 2 seconds — all of that was run and confirmed locally before this guide was written. What only your own dev store can confirm is Step 4's registration call succeeding and a real placed order actually reaching this receiver end to end.

Acceptance checklist

- ☐ Registered at least one webhook (`orders/create`) against your dev store, with `userErrors` empty
- ☐ The receiver correctly verifies the HMAC signature and rejects mismatches, missing headers, and malformed headers, without crashing
- ☐ Triggering the event in your dev store (e.g. placing a test order) hits your receiver and logs the real order
- ☐ The receiver responds with `200` immediately, deferring any heavier work until after the response is sent

Troubleshooting

Symptom	Likely cause	Try this
Every real webhook request gets rejected with <code>401</code> , even though your test curl requests pass	<code>express.json()</code> (or another body parser) ran before this route, so <code>req.body</code> is a parsed object instead of the raw bytes Shopify signed	Confirm <code>express.raw({ type: "application/json" })</code> is the body parser used on this specific route, and that no earlier global <code>app.use(express.json())</code> already consumed the request
The process crashes on an unrelated incoming request (a health check, a scanner)	<code>crypto.timingSafeEqual</code> throws when the header is missing or a different length, and nothing is catching it	Wrap the comparison in the <code>try/catch</code> from Step 3, and treat any thrown error as an invalid signature
<code>register-webhook.js</code> succeeds, but no order ever reaches the receiver	The tunnel URL changed after a restart, or points at the wrong local port	Re-run <code>register-webhook.js</code> with the current ngrok URL from Step 2, and confirm the receiver is listening on the same port the tunnel forwards to
<code>webhookSubscriptionCreate</code> returns a <code>userErrors</code> entry about the topic or scope	The Dev Dashboard app isn't scoped to access the resource the topic covers	Confirm the app has at least <code>read_orders</code> access in its Dev Dashboard configuration
Shopify's webhook eventually stops arriving after working initially	The receiver was slow or erroring often enough that Shopify auto-disabled the subscription after repeated delivery failures	Confirm the <code>200</code> response is sent before any slow work runs, not after — re-register the webhook once fixed
The order log line never appears, even though the response was a real <code>200</code>	The deferred <code>setImmediate</code> callback threw (e.g. <code>JSON.parse</code> failing on an unexpected body shape) after the response had already gone out, so the error never surfaced	Temporarily log inside the callback before the <code>JSON.parse</code> call to confirm what actually arrived

Where This Goes Next

Keep this project. Module 5's event-driven, admin-scoped operations agent builds directly on this same HMAC-verified receiver pattern — it just replaces the `console.log` with a call into an agent that decides what to do about the order.